Josue Gomez

Nicolas Matamoros

Felipe Loera

4. Normalización de la base de datos del reto apoyándose en recursos de IA generativa

Instrucciones

Tomando en cuenta las necesidades del reto que plantearon, desarrolle lo siguiente:

- Presente un esquema conceptual **Versión 2** (diagrama entidad-relación) que represente la información de su proyecto. Este modelo debe cumplir, al menos, con la **tercera forma normal (3FN)**.

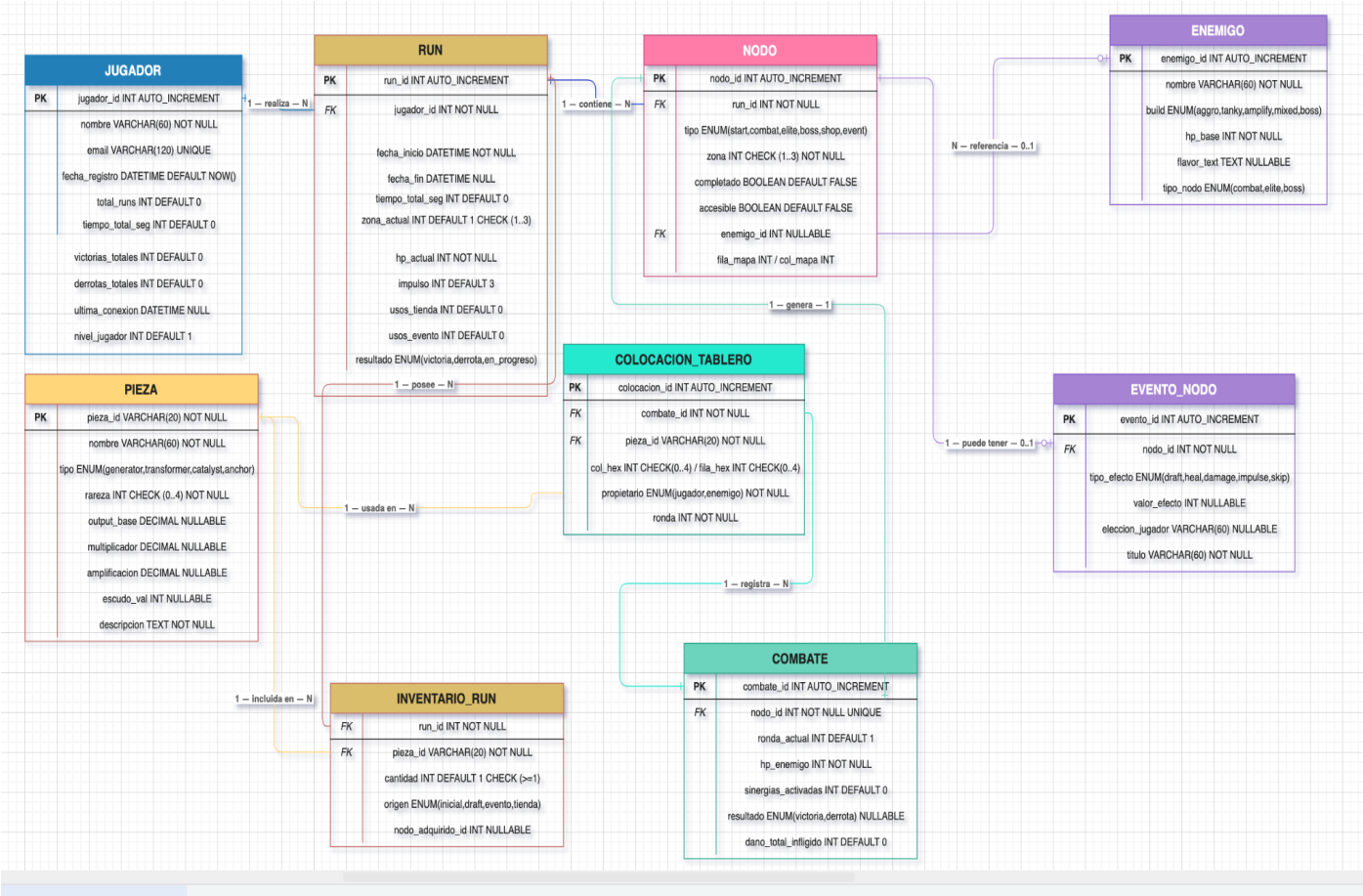
- En el esquema pueden incluir mejoras sugeridas por un modelo generativo. Se les recomienda interactuar con esta herramienta describiendo en texto las entidades y relaciones presentes, y consultarle qué aspectos podrían optimizarse o ajustarse.

- Deben indicar qué elementos fueron normalizados con base en lo aprendido en clase y cuáles fueron modificados siguiendo las recomendaciones del modelo de lenguaje.

- Finalmente, elaboren un documento en **PDF** donde expliquen la importancia de la normalización y cómo las herramientas de lenguaje generativo contribuyeron a mejorar el diagrama entidad-relación, **sin reemplazar el trabajo de los desarrolladores**.

Versión 2 del Diagrama_er

Durante la construcción de la versión 2 del diagrama se realizaron cambios importantes para mejorar la organización de la base de datos, reducir redundancias y facilitar futuras expansiones del proyecto.



Elementos normalizados:

Separación de inventario

Inicialmente, gran parte de la información relacionada con las piezas podía almacenarse directamente dentro de la tabla RUN, lo que generaba redundancia y hacía que el control del inventario fuera un poco más difícil entonces decidimos separarlo en tres entidades:

- RUN
- INVENTARI0_RUN
- PIEZA

De esta forma, cada pieza existe una sola vez en la tabla PIEZA, mientras que INVENTARIO_RUN se encarga de relacionarla con cada partida específica, asi evitando que los datos se puedan duplicar.

Separación de combate

También se mejoró la estructura del sistema de combate, en lugar de guardar toda la información en una sola tabla, se separó en tres que vienen siendo

- COMBATE
- COLOCACION_TABLRO
- ENEMIGO

Para así permitir que cada entidad tenga una sola responsabilidad.

COMBATE guarda el resultado general del enfrentamiento, ENEMIGO almacena la información base de cada enemigo, y COLOCACION_TABLERO registra cómo se colocan las piezas durante cada ronda.

Esto mejora mucho la escalabilidad del sistema.

Eventos independientes

La identidad, EVENTO_NODO fue creada para asi en lugar de guardar toda la información de eventos directamente dentro de NODO.Esto ayuda a separar la lógica de exploración de la lógica de eventos especiales, evitando dependencias parciales y mejorando la claridad del modelo.

Enemigos separados de nodos

Antes, el enemigo podía considerarse como un simple atributo dentro de NODO.

Ahora:

- NODO referencia a ENEMIGO

Esto evita repetir información del mismo enemigo múltiples veces y mejora la integridad referencial.

Cambios incorporados con recomendaciones del modelo generativo

Para esto se hizo una consulta a ChatGPT describiendo la estructura general de nuestro juego **Arcane Assembly** y se solicito sugerencias para mejorar la normalización del diagrama.

Las recomendaciones que fueron aceptadas y que vivimos que podrían hacer nuestra base de datos más útil para nuestro jugeos fueron las siguientes:

Registro del tiempo final de cada run

Gracias a el profesor y a la IA se detectó que la tabla RUN no almacenaba correctamente el momento exacto en que termina una partida ni cuánto tiempo duró.

Por lo tanto agregamos los siguientes atributos:

- fecha_fin
- tiempo_total_seg

Esto nos permitía :

- medir la duración real de cada run
- analizar el rendimiento del jugador
- generar estadísticas de progreso

Además, evita depender únicamente de cálculos externos desde la aplicación.

Mejora de datos globales del jugador

También se sugirió ampliar la entidad JUGADOR para guardar estadísticas generales del progreso del usuario y no solamente información básica.

Se agregaron los siguientes atributos:

- victorias_totales
- derrotas_totales
- ultima_conexion
- nivel_jugador

Esto permite tener y llevar un mejor control del progreso general del jugador, analizar retención de usuarios y registrar estadísticas persistentes fuera de cada run individual. Así se separan correctamente los datos permanentes del jugador de los datos temporales de cada partida.

Separación lógica entre combate y tablero

Se recomendó mantener separadas las tablas:

- COMBATE
- COLOCACION_TABLERO

ya que representan procesos distintos dentro del sistema, COMBATE representa el resultado del enfrentamiento, mientras que COLOCACION_TABLERO registra las decisiones tácticas dentro de cada ronda. Esto mejora la integridad del modelo y facilita futuras expansiones.

También se validó el uso de llaves primarias artificiales como:

- run_id
- combate_id
- colocacion_id

Esto mejora la integridad referencial y facilitar futuras relaciones entre tablas sin depender de llaves compuestas más complejas.

Importancia de la normalización

La normalización es el proceso de organizar una base de datos para reducir redundancia, evitar errores y mejorar la integridad de la información.

En este caso de nuestro videojuego la normalización permitió separar correctamente entidades como Run, Combate, Inventario, Piezas, Nodos, Eventos y Jugador, evitando que una sola tabla concentre demasiada información.

Esto para que la base de datos fuera mucho más clara, más fácil de mantener y preparada para futuras mejoras del proyecto.

Los principales beneficios fueron:

- **Consistencia de datos:** cada dato se almacena una sola vez.
- **Reducción de errores:** se evitan problemas al actualizar, insertar o eliminar información.
- **Escalabilidad:** facilita agregar nuevas mecánicas sin rehacer toda la base de datos.
- **Integridad referencial:** las llaves foráneas aseguran relaciones correctas entre tablas.
- **Mejor análisis del juego:** permite medir duración de runs, progreso del jugador, combates ganados y estadísticas generales.

Además, la normalización nos permite separar correctamente los datos permanentes del jugador de los datos temporales de cada partida.

Mientras la tabla JUGADOR almacena estadísticas globales como victorias totales y tiempo acumulado, la tabla RUN registra únicamente la información específica de cada partida.Esto para evitar redundancias y mejorar el análisis general del sistema.

Rol de la IA generativa en el proceso

Las herramientas de IA generativa se utilizaron correcta y únicamente como apoyo técnico y no como reemplazo del trabajo de los desarrolladores.

La función principal de la IA fue ayudar a revisar la estructura del diagrama, detectar posibles redundancias y sugerir buenas prácticas de modelado.

Entre sus aportaciones estuvieron:

- detectar campos que faltaban, como el tiempo final de cada run
- sugerir una mejor separación entre entidades relacionadas
- validar el uso correcto de llaves primarias y foráneas
- proponer mejoras para el registro de estadísticas del jugador

En todos los casos, cada sugerencia fue revisada por el equipo antes de implementarse. La IA ayudó principalmente a acelerar el proceso de revisión y a identificar detalles que podrían mejorarse, pero las decisiones finales siempre fueron tomadas por nosotros, los desarrolladores del proyecto.